

Advanced Artificial Intelligence and Machine Learning

DL Intro I



Learning goals

- Historical development of neural networks, from early threshold units to modern foundation models
- The single neuron (perceptron) as an affine transformation plus non-linear activation, its hypothesis space, and typical loss functions
- Single- and multi-hidden-layer feedforward networks, representation learning, and common activation functions
- Compact vector/matrix notation for feedforward network computations
- Softmax output layers and loss functions for multi-class classification
- The universal approximation theorem and its implications for model flexibility

Brief History

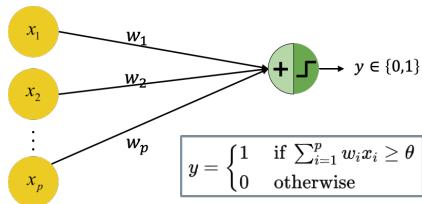
A BRIEF HISTORY: WHAT WAS POSSIBLE

- 1943: McCulloch & Pitts propose the “Threshold Logic Unit (TLU)”, the first artificial neuron: binary inputs, a fixed threshold θ , non-adjustable weights.
- 1957: Rosenblatt’s Perceptron generalizes this with real-valued inputs and, crucially, **learnable** weights.
- 1960: Widrow & Hoff’s ADALINE learns by adjusting weights based on the weighted sum of the inputs.
- 1965: Ivakhnenko’s Group Method of Data Handling (GMDH) gives the first learning algorithm for supervised, deep, multilayer feedforward networks.

A BRIEF HISTORY: WHAT WAS POSSIBLE

/ 2

- 1985: Rumelhart, Hinton & Williams popularize **backpropagation** for multilayer perceptrons: efficient gradient computation through composite functions (first derived by Linnainmaa already in 1970 – more on this in a later session).



A BRIEF HISTORY: WHY THE WINTERS

- 1969 — First AI Winter: Minsky & Papert prove that a single perceptron cannot solve the XOR problem: it is not linearly separable (we return to this in detail later). Funding dries up, research stalls.
- 1985 — Second AI Winter: Despite backpropagation, expectations outpace results. Kernel machines and graphical models match or beat early neural nets on many tasks, and modeling long sequences remains a fundamental difficulty. “AI” becomes associated with pseudoscience; investment collapses again.

A BRIEF HISTORY: WHY THE WINTERS

/ 2

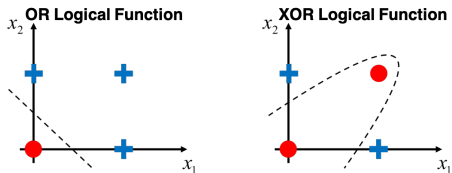


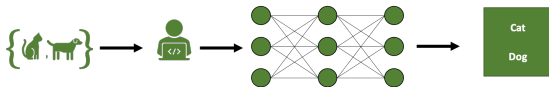
Figure: The XOR problem: not linearly separable.

A BRIEF HISTORY OF NEURAL NETWORKS

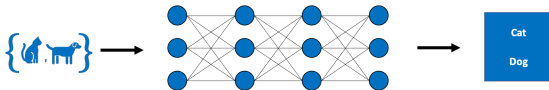
- 2006: Age of deep neural networks began.
 - Geoffrey Hinton showed that a deep belief network could be efficiently trained using *greedy layer-wise pretraining*.
 - This wave of research popularized the use of the term deep learning to emphasize that researchers were now able to train deeper neural networks than had been possible before.
 - At this time, deep neural networks outperformed competing AI systems based on other ML technologies as well as hand-designed functionality.

A BRIEF HISTORY OF NEURAL NETWORKS / 2

Machine Learning

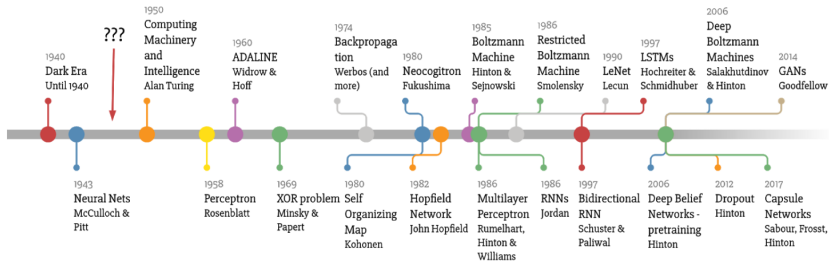


Deep Learning



A BRIEF HISTORY OF NEURAL NETWORKS / 3

Deep Learning Timeline

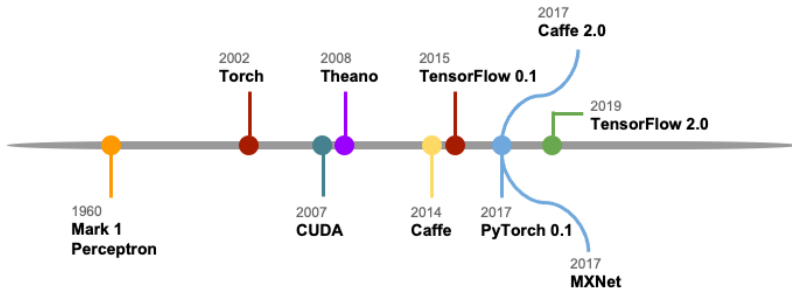


A BRIEF HISTORY OF NEURAL NETWORKS / 4

Figure: (Vazquez, 2018)

A BRIEF HISTORY OF NEURAL NETWORKS / 5

History of DL Tools



WHAT THE FIELD LOOKS LIKE TODAY

- A major shift in the 2020s was the move from highly specialized models to **foundation models**: large pretrained models that can be adapted to many downstream tasks.
- The frontier is increasingly **multimodal**: models can jointly process text, images, audio, video, and actions.
- Modern systems are also becoming more **interactive**: they can write code, search, retrieve documents, and call external tools.
- Examples of this broader landscape include **GPT-5** and **Claude Opus** for general-purpose assistants, **Stable Diffusion** for image generation, **DINOv3** for self-supervised visual representations, and **TabPFN** for tabular prediction.

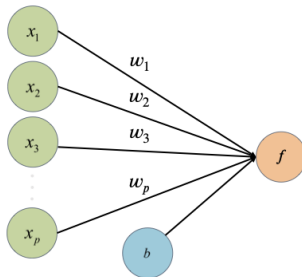
WHAT THE FIELD LOOKS LIKE TODAY / 2

- Progress is now often driven by a combination of
 - better architectures and training objectives,
 - much larger datasets,
 - more compute and specialized hardware,
 - stronger software tooling and engineering.

The modern DL pipeline is frequently: **pretrain at scale, then adapt with prompting, finetuning, retrieval, or tool use.**

Single Neuron / Perceptron

A SINGLE NEURON



Perceptron with **input features** $x_1, x_2, \dots, x_p$, **weights** $w_1, w_2, \dots, w_p$, **bias term** b , and **activation function** τ .

- The perceptron is a single artificial neuron and the basic computational unit of neural networks.

A SINGLE NEURON / 2

- It is a weighted sum of input values, transformed by τ :

$$f(\mathbf{x}) = \tau(w_1x_1 + \dots + w_px_p + b) = \tau(\mathbf{w}^T\mathbf{x} + b)$$

A SINGLE NEURON

Activation function τ : a single neuron represents different functions depending on the choice of activation function.

- The identity function gives us the simple **linear regression**:

$$f(x) = \tau(w^T x) = w^T x$$

- The logistic function gives us the **logistic regression**:

$$f(x) = \tau(w^T x) = \frac{1}{1 + \exp(-w^T x)}$$

A SINGLE NEURON

We consider a perceptron with 3-dimensional input, i.e.

$$f(\mathbf{x}) = \tau(w_1 \mathbf{x}_1 + w_2 \mathbf{x}_2 + w_3 \mathbf{x}_3 + b).$$

- Input features $\mathbf{x}$ are represented by nodes in the “input layer”.

Input

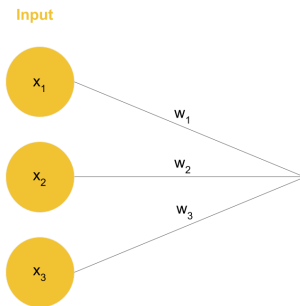


A SINGLE NEURON / 2

- In general, a p -dimensional input vector x will be represented by p nodes in the input layer.

A SINGLE NEURON / 3

- Weights w are connected to edges from the input layer.



- The bias term b is implicit here. It is often not visualized as a separate node.

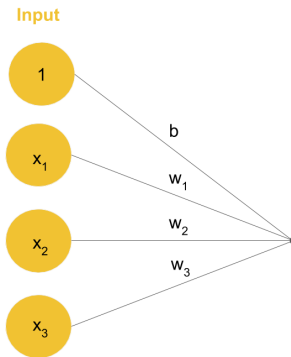
A SINGLE NEURON / 4

For an explicit graphical representation, we do a simple trick:

- Add a constant feature to the inputs $\tilde{\mathbf{x}} = (1, x_1, \dots, x_p)^T$
- and absorb the bias into the weight vector $\tilde{\mathbf{w}} = (b, w_1, \dots, w_p)$.

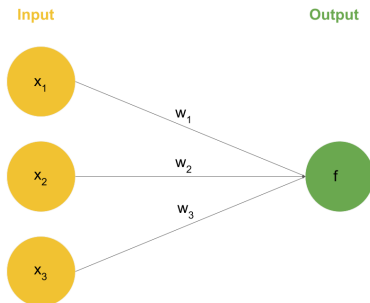
The graphical representation is then:

A SINGLE NEURON / 5



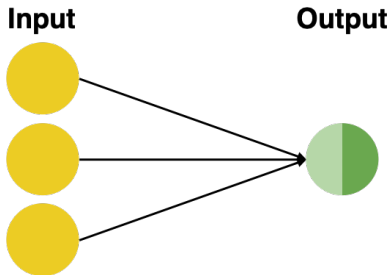
A SINGLE NEURON / 6

- The computation $\tau(w_1x_1 + w_2x_2 + w_3x_3 + b)$ is represented by the neuron in the “output layer”.



A SINGLE NEURON / 7

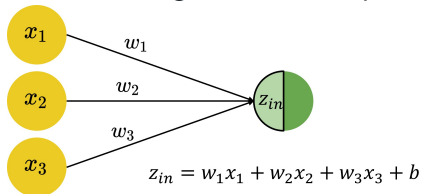
- You can picture the input vector being "fed" to neurons on the left followed by a sequence of computations performed from left to right. This is called a **forward pass**.



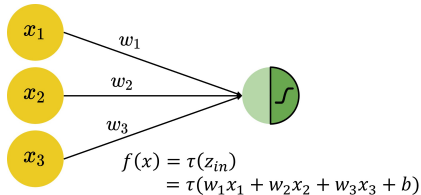
A SINGLE NEURON

A neuron performs a 2-step computation:

- 1 **Affine Transformation:** weighted sum of inputs plus bias.



- 2 **Non-linear Activation:** a non-linear transformation applied to the weighted sum.



A SINGLE NEURON: HYPOTHESIS SPACE

- The hypothesis space that is formed by single neuron is

$$\mathcal{H} = \left\{ f : \mathbb{R}^p \rightarrow \mathbb{R} \mid f(\mathbf{x}) = \tau \left(\sum_{j=1}^p w_j x_j + b \right), w \in \mathbb{R}^p, b \in \mathbb{R} \right\}.$$

- If τ is the logistic sigmoid or identity function, $\mathcal{H}$ corresponds to the hypothesis space of logistic or linear regression, respectively.

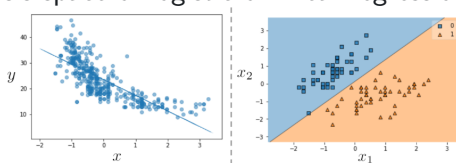


Figure: *Left:* A regression line learned by a single neuron. *Right:* A decision-boundary learned by a single neuron in a binary classification task.

A SINGLE NEURON: OPTIMIZATION

- To optimize this model, we minimize the empirical risk

$$\mathcal{R}_{\text{emp}} = \frac{1}{n} \sum_{i=1}^n L \left(y^{(i)}, f \left(\mathbf{x}^{(i)} \right) \right),$$

where $L(y, f(\mathbf{x}))$ is a loss function. It compares the network's predictions $f(\mathbf{x})$ to the ground truth y .

- For regression, we typically use the L2 loss (rarely L1):

$$L(y, f(\mathbf{x})) = \frac{1}{2}(y - f(\mathbf{x}))^2$$

- For binary classification, we typically apply the cross entropy loss (also known as Bernoulli loss):

$$L(y, f(\mathbf{x})) = -(y \log f(\mathbf{x}) + (1 - y) \log(1 - f(\mathbf{x})))$$

A SINGLE NEURON: OPTIMIZATION / 2

- For a single neuron and both choices of τ the loss function is convex.
- The global optimum can be found with an iterative algorithm like gradient descent.
- A single neuron with logistic sigmoid function trained with the Bernoulli loss yields the same result as logistic regression when trained until convergence.
- Note: In the case of regression and the L2-loss, the solution can also be found analytically using the “normal equations”. However, in other cases a closed-form solution is usually not available.

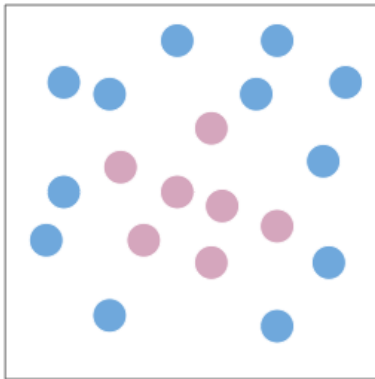
Single Hidden Layer Neural Networks

MOTIVATION

- The graphical way of representing simple functions/models, like logistic regression. Why is that useful?
- Because individual neurons can be used as building blocks of more complicated functions.
- Networks of neurons can represent extremely complex hypothesis spaces.
- Most importantly, it allows us to define the “right” kinds of hypothesis spaces to learn functions that are common in our universe in a data-efficient way (see Lin, Tegmark et al. 2016).

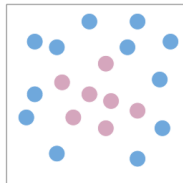
MOTIVATION / 2

Can a single neuron perform binary classification of these points?



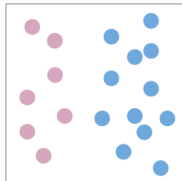
MOTIVATION / 3

- As a single neuron is restricted to learning only linear decision boundaries, its performance on the following task is quite poor:



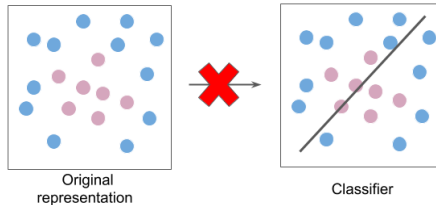
- However, the neuron can easily separate the classes if the original features are transformed (e.g., from Cartesian to polar coordinates):

MOTIVATION / 4



MOTIVATION / 5

- Instead of classifying the data in the original representation,



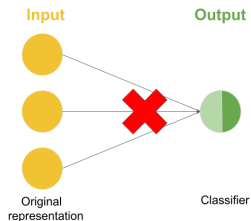
- we classify it in a new feature space.

MOTIVATION / 6



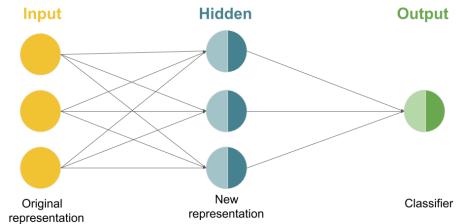
MOTIVATION / 7

- Analogously, instead of a single neuron,



- we use more complex networks.

MOTIVATION / 8



REPRESENTATION LEARNING

- It is *very* critical to feed a classifier the “right” features in order for it to perform well.
- Before deep learning took off, features for tasks like machine vision and speech recognition were “hand-designed” by domain experts. This step of the machine learning pipeline is called **feature engineering**.
- DL automates feature engineering. This is called **representation learning**.

SINGLE HIDDEN LAYER NETWORKS

Single neurons perform a 2-step computation:

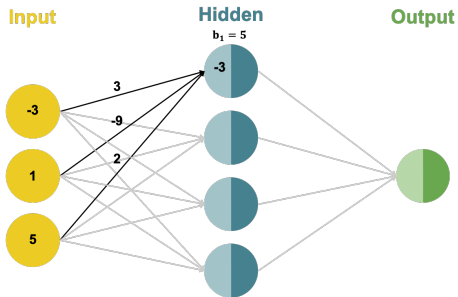
- ① **Affine Transformation:** a weighted sum of inputs plus bias.
- ② **Activation:** a non-linear transformation on the weighted sum.

Single hidden layer networks consist of two layers (without input layer):

- ① **Hidden Layer:** having a set of neurons.
 - ② **Output Layer:** having one or more output neurons.
- Multiple inputs are simultaneously fed to the network.
 - Each neuron in the hidden layer performs a 2-step computation.
 - The final output of the network is then calculated by another 2-step computation performed by the neuron in the output layer.

SINGLE HIDDEN LAYER NETWORKS: EXAMPLE

Each neuron in the hidden layer performs an **affine transformation** on the inputs:

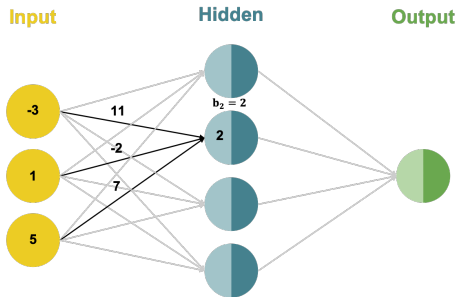


$$z_{1,in} = w_{11}x_1 + w_{21}x_2 + w_{31}x_3 + b_1$$

$$z_{1,in} = 3 * (-3) + (-9) * 1 + 2 * 5 + 5 = -3$$

SINGLE HIDDEN LAYER NETWORKS: EXAMPLE

Each neuron in the hidden layer performs an **affine transformation** on the inputs:

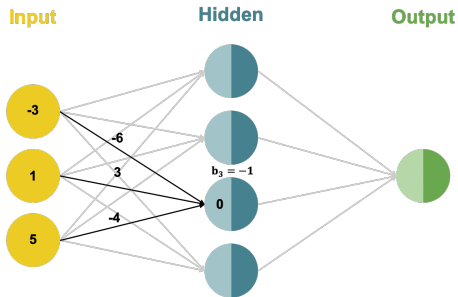


$$z_{2,\text{in}} = w_{12}x_1 + w_{22}x_2 + w_{32}x_3 + b_2$$

$$z_{2,\text{in}} = 11 * (-3) + (-2) * 1 + 7 * 5 + 2 = 2$$

SINGLE HIDDEN LAYER NETWORKS: EXAMPLE

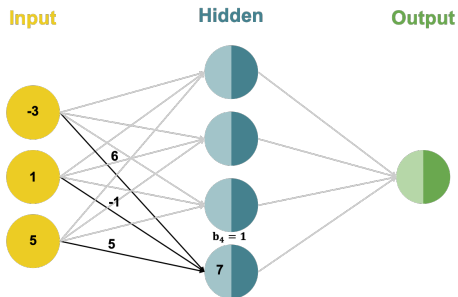
Each neuron in the hidden layer performs an **affine transformation** on the inputs:



$$z_{3,\text{in}} = w_{13}x_1 + w_{23}x_2 + w_{33}x_3 + b_3$$
$$z_{3,\text{in}} = (-6) * (-3) + 3 * 1 + (-4) * 5 - 1 = 0$$

SINGLE HIDDEN LAYER NETWORKS: EXAMPLE

Each neuron in the hidden layer performs an **affine transformation** on the inputs:

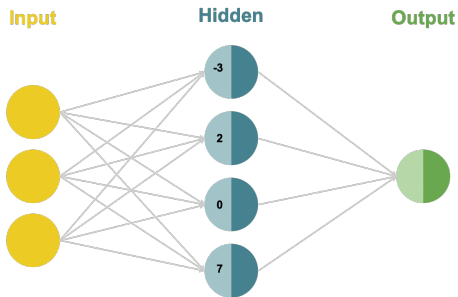


$$z_{4,\text{in}} = w_{14}x_1 + w_{24}x_2 + w_{34}x_3 + b_4$$

$$z_{4,\text{in}} = 6 * (-3) + (-1) * 1 + 5 * 5 + 1 = 7$$

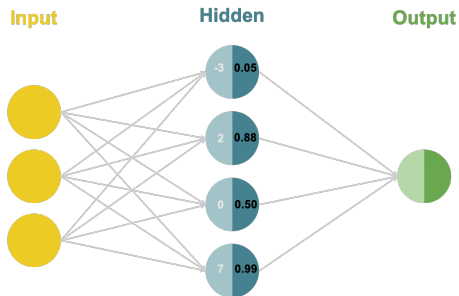
SINGLE HIDDEN LAYER NETWORKS: EXAMPLE

Each neuron in the hidden layer performs an **affine transformation** on the inputs:



SINGLE HIDDEN LAYER NETWORKS: EXAMPLE

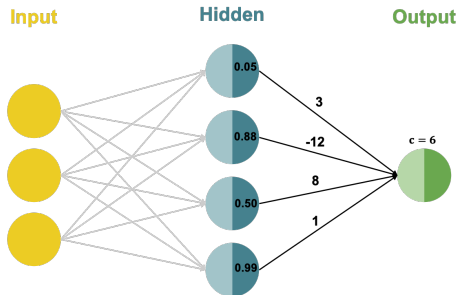
Each hidden neuron performs a non-linear **activation** transformation on the weight sum:



$$z_{i,\text{out}} = \sigma(z_{i,\text{in}}) = \frac{1}{1 + e^{-z_{i,\text{in}}^{(i)}}}$$

SINGLE HIDDEN LAYER NETWORKS: EXAMPLE

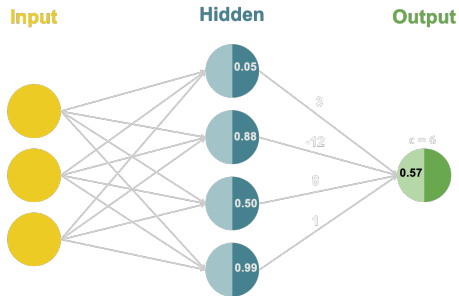
The output neuron performs an **affine transformation** on its inputs:



$$f_{in} = u_1 z_{1,out} + u_2 z_{2,out} + u_3 z_{3,out} + u_4 z_{4,out} + C$$

SINGLE HIDDEN LAYER NETWORKS: EXAMPLE

The output neuron performs an **affine transformation** on its inputs:

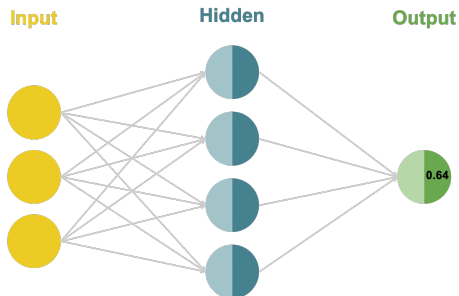


$$f_{\text{in}} = u_1 z_{1,\text{out}} + u_2 z_{2,\text{out}} + u_3 z_{3,\text{out}} + u_4 z_{4,\text{out}} + c$$

$$f_{\text{in}} = 3 * 0.05 + (-12) * 0.88 + 8 * 0.50 + 1 * 0.99 + 6 = 0.57$$

SINGLE HIDDEN LAYER NETWORKS: EXAMPLE

The output neuron performs a non-linear **activation** transformation on the weight sum:



$$f_{\text{out}} = \sigma(f_{\text{in}}) = \frac{1}{1+e^{-f_{\text{in}}}}$$
$$f_{\text{out}} = \frac{1}{1+e^{-0.57}} = 0.64$$

HIDDEN LAYER: ACTIVATION FUNCTION

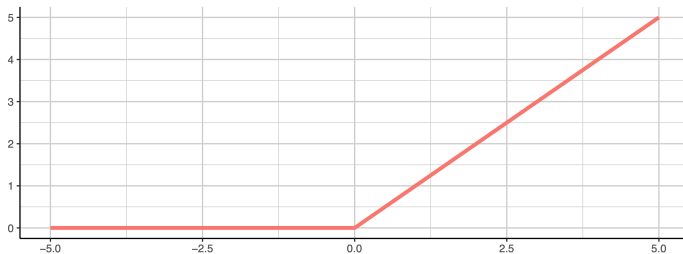
- If the hidden layer does not have a non-linear activation, the network can only learn linear decision boundaries.
- A lot of different activation functions exist.

HIDDEN LAYER: ACTIVATION FUNCTION

ReLU Activation:

- Currently the most popular choice is the ReLU (rectified linear unit):

$$\sigma(v) = \max(0, v)$$



HIDDEN LAYER: ACTIVATION FUNCTION

/ 2

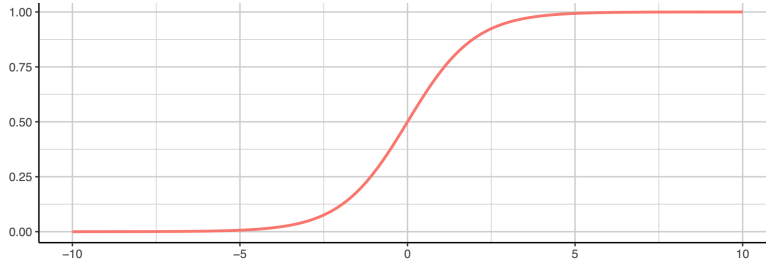
Sigmoid Activation Function:

- The sigmoid function can be used even in the hidden layer:

$$\sigma(v) = \frac{1}{1 + \exp(-v)}$$

HIDDEN LAYER: ACTIVATION FUNCTION

/ 3



Matrix Notation

VECTORIZED NOTATION: HIDDEN LAYER

- Input $\mathbf{x} \in \mathbb{R}^{p \times 1}$, weight matrix $\mathbf{W} \in \mathbb{R}^{p \times m}$ (one column per hidden neuron), bias $\mathbf{b} \in \mathbb{R}^{m \times 1}$.
- All m hidden neurons $z_1, \dots, z_m$ are computed at once:

$$z_{in} = \mathbf{W}^T \mathbf{x} + \mathbf{b}, \quad \mathbf{z} = \sigma(\mathbf{z}_{in}) = \sigma(\mathbf{W}^T \mathbf{x} + \mathbf{b}),$$

where σ is applied element-wise.

- **Note:** The bias is often folded into $\mathbf{W}$ by augmenting $\mathbf{x}$ with a constant 1; we won't draw it explicitly from here on.

VECTORIZED NOTATION: OUTPUT LAYER

- For regression or binary classification: one output unit f where
 - $f_{in} = \mathbf{u}^T \mathbf{z} + c$, i.e. a linear combination of derived features plus the bias term c of the output neuron, and
 - $f(\mathbf{x}) = f_{out} = \tau(f_{in}) = \tau(\mathbf{u}^T \mathbf{z} + c)$, where τ is the output activation function.
- For regression τ is the identity function; for binary classification, τ is a sigmoid function.
- **Note:** σ introduces the non-linearities that let the network learn complex functions; τ merely rescales the final score to match the target's range.

VECTORIZED NOTATION: BATCHED INPUTS

- Multiple inputs $\mathbf{x}^{(i)}$, $i \in \{1, \dots, n\}$, are stacked as rows of a **design matrix** $\mathbf{X} \in \mathbb{R}^{n \times p}$.
- Hidden activations for the whole batch, computed at once:

$$\mathbf{Z} = \sigma(\mathbf{X}\mathbf{W} + \mathbf{B}) \in \mathbb{R}^{n \times m},$$

where $\mathbf{B}$ duplicates $\mathbf{b}$ as the rows of an $(n \times m)$ matrix.

- Final output for the whole batch:

$$\tau(\mathbf{Z}\mathbf{u} + \mathbf{C}) \in \mathbb{R}^{n \times 1},$$

where $\mathbf{C}$ duplicates the scalar bias c as the rows of an $(n \times 1)$ matrix.

The XOR Problem

EXAMPLE: XOR PROBLEM

- Suppose we have four data points

$$X = \{(0, 0)^T, (0, 1)^T, (1, 0)^T, (1, 1)^T\}$$

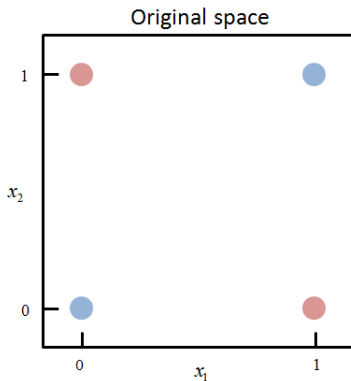
- The XOR gate (exclusive or) returns true, when an odd number of inputs are true:

x_1	x_2	XOR = y
0	0	0
0	1	1
1	0	1
1	1	0

- Can you learn the target function with a logistic regression model?

EXAMPLE: XOR PROBLEM / 2

- Logistic regression can not solve this problem. In fact, any model using simple hyperplanes for separation can not (including a single neuron).
- A small neural net can easily solve the problem by transforming the space!



EXAMPLE: XOR PROBLEM / 3

- Consider the following model:

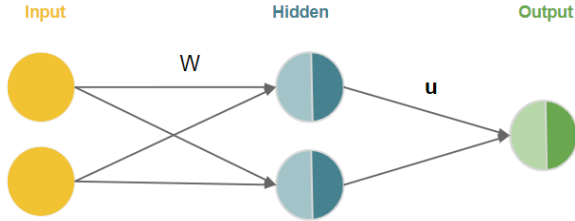


Figure: A neural network with two neurons in the hidden layer. The matrix W describes the mapping from x to z . The vector u from z to y .

EXAMPLE: XOR PROBLEM / 4

- Let use ReLU $\sigma(z) = \max\{0, z\}$ as activation function and a simple thresholding function $\tau(z) = [z > 0] = \begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{otherwise} \end{cases}$ as output transformation function. We can represent the architecture of the model by the following equation:

$$\begin{aligned} f(x \mid W, b, u, c) &= \tau\left(u^\top \sigma(W^\top x + b) + c\right) \\ &= \tau\left(u^\top \max\{0, W^\top x + b\} + c\right) \end{aligned}$$

- So how many parameters does our model have?

EXAMPLE: XOR PROBLEM / 5

- In a fully connected neural net, the number of connections between the nodes equals our parameters:

$$\underbrace{(2 \times 2)}_W + \underbrace{(2 \times 1)}_b + \underbrace{(2 \times 1)}_u + \underbrace{(1)}_c = 9$$

EXAMPLE: XOR PROBLEM / 6

$$\text{Let } W = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}, \quad b = \begin{pmatrix} 0 \\ -1 \end{pmatrix}, \quad u = \begin{pmatrix} 1 \\ -2 \end{pmatrix}, \quad c = -0.5$$

$$X = \begin{pmatrix} 0 & 0 \\ 0 & 1 \\ 1 & 0 \\ 1 & 1 \end{pmatrix}, \quad XW = \begin{pmatrix} 0 & 0 \\ 1 & 1 \\ 1 & 1 \\ 2 & 2 \end{pmatrix}, \quad XW + \mathbf{B} = \begin{pmatrix} 0 & -1 \\ 1 & 0 \\ 1 & 0 \\ 2 & 1 \end{pmatrix}$$

Note: X is a $(n \times p)$ design matrix in which the *rows* correspond to the data points. W , as usual, is a $(p \times m)$ matrix where each *column* corresponds to a single (hidden) neuron. $\mathbf{B}$ is a $(n \times m)$ matrix with b duplicated along the rows.

EXAMPLE: XOR PROBLEM / 7

$$X = \begin{pmatrix} 0 & 0 \\ 0 & 1 \\ 1 & 0 \\ 1 & 1 \end{pmatrix} \quad W = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$$

EXAMPLE: XOR PROBLEM / 8

$$\text{Let } W = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}, b = \begin{pmatrix} 0 \\ -1 \end{pmatrix}, u = \begin{pmatrix} 1 \\ -2 \end{pmatrix}, c = -0.5$$

$$X = \begin{pmatrix} 0 & 0 \\ 0 & 1 \\ 1 & 0 \\ 1 & 1 \end{pmatrix}, XW = \begin{pmatrix} 0 & 0 \\ 1 & 1 \\ 1 & 1 \\ 2 & 2 \end{pmatrix}, XW + \mathbf{B} = \begin{pmatrix} 0 & -1 \\ 1 & 0 \\ 1 & 0 \\ 2 & 1 \end{pmatrix}$$

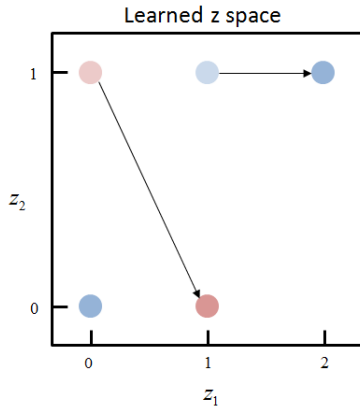
$$\mathbf{Z} = \max\{0, XW + \mathbf{B}\} = \begin{pmatrix} 0 & 0 \\ 1 & 0 \\ 1 & 0 \\ 2 & 1 \end{pmatrix}$$

- Note that we computed all examples at once.

EXAMPLE: XOR PROBLEM / 9

- The input points are mapped into transformed space to

$$\mathbf{Z} = \begin{pmatrix} 0 & 0 \\ 1 & 0 \\ 1 & 0 \\ 2 & 1 \end{pmatrix}$$

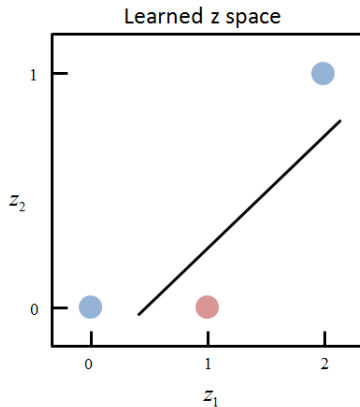


EXAMPLE: XOR PROBLEM / 10

- The input points are mapped into transformed space to

$$\mathbf{Z} = \begin{pmatrix} 0 & 0 \\ 1 & 0 \\ 1 & 0 \\ 2 & 1 \end{pmatrix}$$

which is easily separable.



EXAMPLE: XOR PROBLEM / 11

- In a final step we have to multiply the activated values of matrix $\mathbf{Z}$ with the vector $\mathbf{u}$ and add the bias term c :

$$f(x \mid W, b, u, c) = \begin{pmatrix} 0 & 0 \\ 1 & 0 \\ 1 & 0 \\ 2 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ -2 \end{pmatrix} + \begin{pmatrix} -0.5 \\ -0.5 \\ -0.5 \\ -0.5 \end{pmatrix} = \begin{pmatrix} -0.5 \\ 0.5 \\ 0.5 \\ -0.5 \end{pmatrix}$$

- And then apply the step function $\tau(z) = [z > 0]$. This solves the XOR problem perfectly!

x_1	x_2	XOR = y
0	0	0
0	1	1
1	0	1
1	1	0

NEURAL NETWORKS : OPTIMIZATION

- In this simple example we actually “guessed” the values of the parameters for $\mathbf{W}$, b , u and c .
- That won't work for more sophisticated problems!
- We will learn later about iterative optimization algorithms for automatically adapting weights and biases.
- An added complication is that the loss function is no longer convex. Therefore, there might not exist a single minimum.

Multi-Class Classification

MULTI-CLASS CLASSIFICATION

- We have only considered regression and binary classification problems so far.
- How can we get a neural network to perform multiclass classification?

MULTI-CLASS CLASSIFICATION

- The first step is to add additional neurons to the output layer.
- Each neuron in the layer will represent a specific class (number of neurons in the output layer = number of classes).

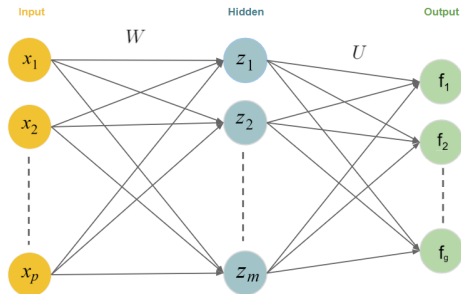


Figure: Structure of a single hidden layer, feed-forward neural network for g -class classification problems (bias term omitted).

MULTI-CLASS CLASSIFICATION

Notation:

- For g -class classification, g output units:

$$\mathbf{f} = (f_1, \dots, f_g)$$

- m hidden neurons $z_1, \dots, z_m$, with

$$z_j = \sigma(\mathbf{W}_j^T \mathbf{x}), \quad j = 1, \dots, m.$$

- Compute linear combinations of derived features $\mathbf{z}$:

$$f_{in,k} = \mathbf{U}_k^T \mathbf{z}, \quad \mathbf{z} = (z_1, \dots, z_m)^T, \quad k = 1, \dots, g$$

MULTI-CLASS CLASSIFICATION

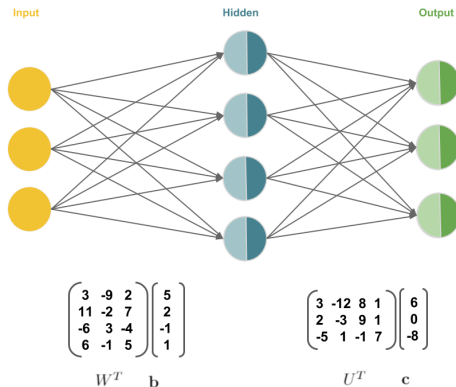
- The second step is to apply a **softmax** activation function to the output layer.
- This gives us a probability distribution over g different possible classes:

$$f_{out,k} = \tau_k(f_{in,k}) = \frac{\exp(f_{in,k})}{\sum_{k'=1}^g \exp(f_{in,k'})}$$

- This is the same transformation used in softmax regression!
- Derivative $\frac{\partial \tau(f_{in})}{\partial f_{in}} = \text{diag}(\tau(f_{in})) - \tau(f_{in})\tau(f_{in})^T$
- It is a “smooth” approximation of the argmax operation, so $\tau((1, 1000, 2)^T) \approx (0, 1, 0)^T$ (picks out 2nd element!).

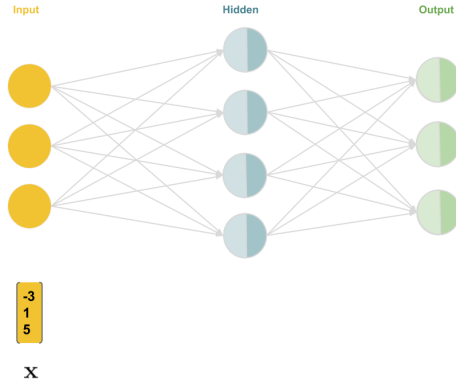
MULTI-CLASS CLASSIFICATION: EXAMPLE

Forward pass (Hidden: Sigmoid, Output: Softmax).



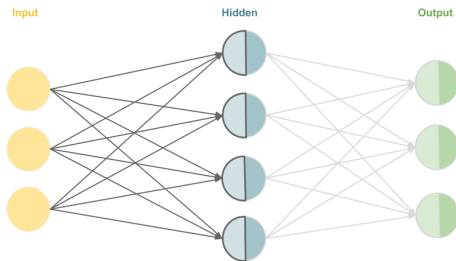
MULTI-CLASS CLASSIFICATION: EXAMPLE

Forward pass (Hidden: Sigmoid, Output: Softmax).



MULTI-CLASS CLASSIFICATION: EXAMPLE

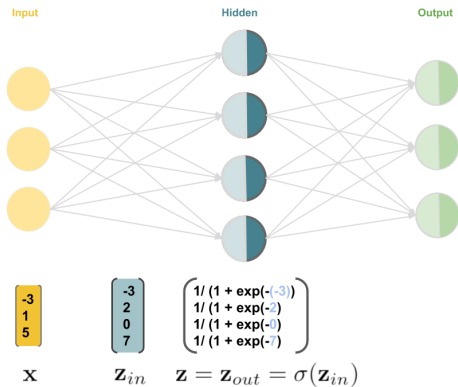
Forward pass (Hidden: Sigmoid, Output: Softmax).



$$\mathbf{x} = \begin{bmatrix} -3 \\ 1 \\ 5 \end{bmatrix} \quad \mathbf{z}_{in} = \mathbf{w}^T \mathbf{x} + \mathbf{b}$$
$$\begin{pmatrix} (-3)*3 + 1*(-9) + 5*2 + 5 \\ (-3)*11 + 1*(-2) + 5*7 + 2 \\ (-3)*(-6) + 1*3 + 5*(-4) + (-1) \\ (-3)*6 + 1*(-1) + 5*5 + 1 \end{pmatrix}$$

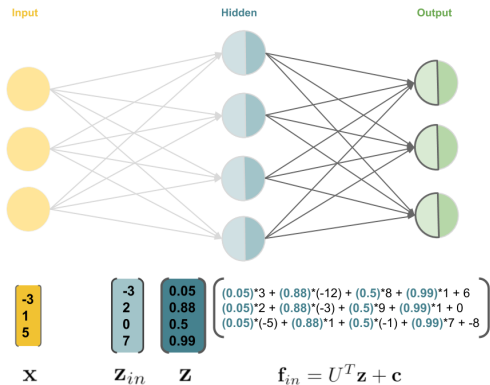
MULTI-CLASS CLASSIFICATION: EXAMPLE

Forward pass (Hidden: Sigmoid, Output: Softmax).



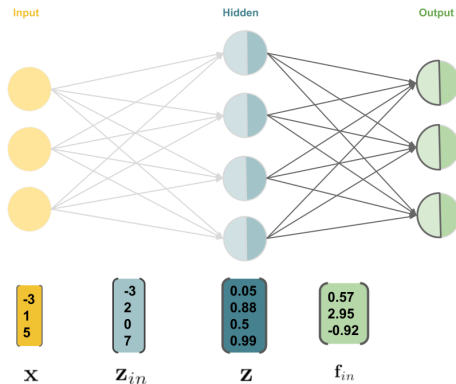
MULTI-CLASS CLASSIFICATION: EXAMPLE

Forward pass (Hidden: Sigmoid, Output: Softmax).



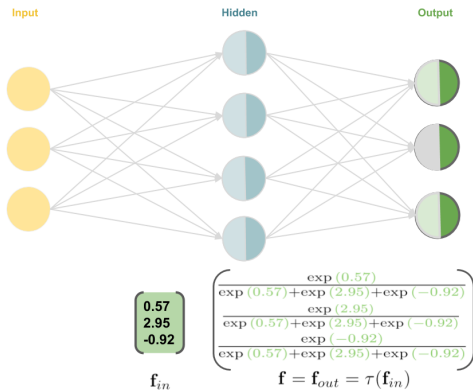
MULTI-CLASS CLASSIFICATION: EXAMPLE

Forward pass (Hidden: Sigmoid, Output: Softmax).



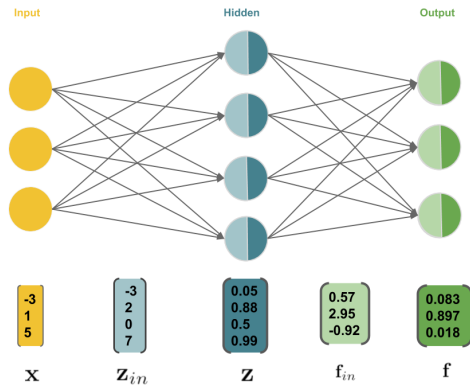
MULTI-CLASS CLASSIFICATION: EXAMPLE

Forward pass (Hidden: Sigmoid, Output: Softmax).



MULTI-CLASS CLASSIFICATION: EXAMPLE

Forward pass (Hidden: Sigmoid, Output: Softmax).



OPTIMIZATION: SOFTMAX LOSS

- The loss function for a softmax classifier is

$$L(y, f(x)) = - \sum_{k=1}^g [y = k] \log \left(\frac{\exp(f_{in,k})}{\sum_{k'=1}^g \exp(f_{in,k'})} \right)$$

$$\text{where } [y = k] = \begin{cases} 1 & \text{if } y = k \\ 0 & \text{otherwise} \end{cases}.$$

- This is equivalent to the cross-entropy loss when the label vector $\mathbf{y}$ is one-hot coded (e.g. $\mathbf{y} = (0, 0, 1, 0)^T$).
- Optimization: Again, there is no analytic solution.

Multi-Layer Feedforward Neural Networks

FEEDFORWARD NEURAL NETWORKS

- We will now extend the model class once again, such that we allow an arbitrary amount l of hidden layers.
- The general term for this model class is (multi-layer) **feedforward networks** (inputs are passed through the network from left to right, no feedback-loops are allowed)

FEEDFORWARD NEURAL NETWORKS / 2

- We can characterize those models by the following chain structure:

$$f = \tau \circ \phi \circ \sigma^{(l)} \circ \phi^{(l)} \circ \sigma^{(l-1)} \circ \phi^{(l-1)} \circ \dots \circ \sigma^{(1)} \circ \phi^{(1)}$$

where $\sigma^{(i)}$ and $\phi^{(i)}$ are the activation function and the weighted sum of hidden layer i , respectively. τ and ϕ are the corresponding components of the output layer.

- Each hidden layer has:
 - an associated weight matrix $W^{(i)}$, bias $b^{(i)}$, and activations $z^{(i)}$ for $i \in \{1 \dots l\}$.
 - $z^{(i)} = \sigma^{(i)}(\phi^{(i)}) = \sigma^{(i)}(W^{(i)T}z^{(i-1)} + b^{(i)})$, where $z^{(0)} = x$.

FEEDFORWARD NEURAL NETWORKS / 3

- Again, without non-linear activations in the hidden layers, the network can only learn linear decision boundaries.

FEEDFORWARD NEURAL NETWORKS / 4

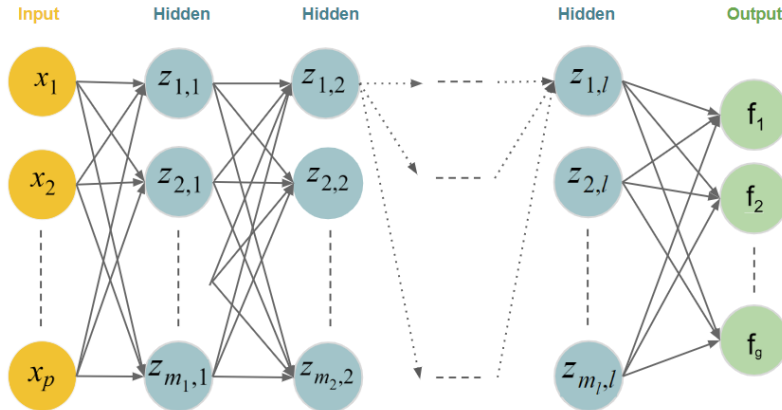
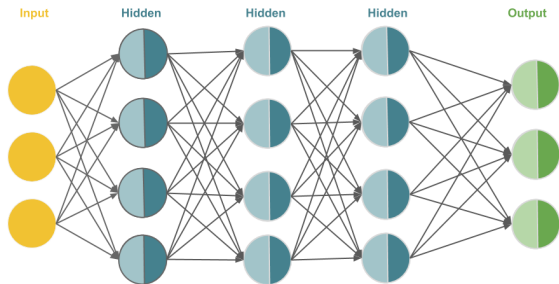


Figure: Structure of a deep neural network with l hidden layers (bias terms omitted).

FEEDFORWARD NEURAL NETWORKS / 5

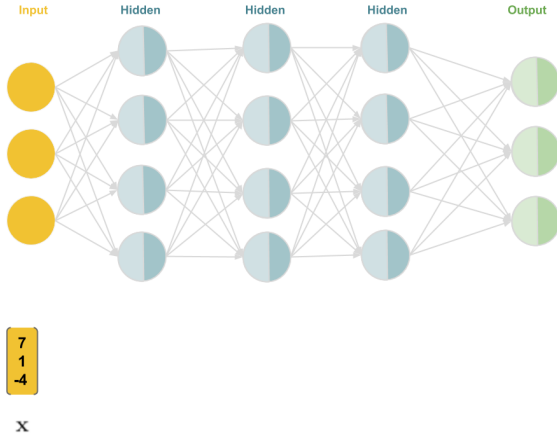
FEEDFORWARD NEURAL NETWORKS: EXAMPLE



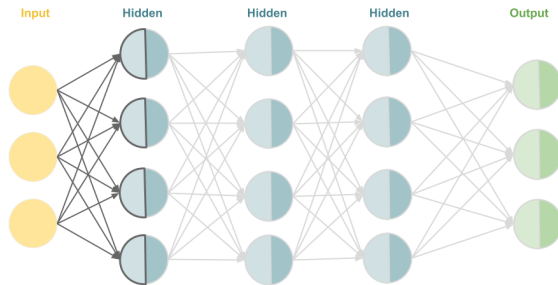
$$\begin{pmatrix} 13 & -9 & 2 \\ -8 & 0 & 3 \\ 4 & -1 & 5 \\ -3 & 12 & 7 \end{pmatrix} \begin{pmatrix} 5 \\ -2 \\ 2 \\ 11 \end{pmatrix} \quad \begin{pmatrix} 1 & 0 & -4 & 1 \\ 0 & 11 & 2 & -14 \\ -1 & 5 & -2 & 16 \\ 0 & -9 & -3 & 4 \end{pmatrix} \begin{pmatrix} -5 \\ 3 \\ 1 \\ -8 \end{pmatrix} \quad \begin{pmatrix} 1 & -2 & -18 & -7 \\ 3 & -4 & 8 & 0 \\ -2 & 1 & 21 & 5 \\ 2 & -2 & 11 & -13 \end{pmatrix} \begin{pmatrix} 4 \\ -6 \\ 1 \\ -17 \end{pmatrix} \quad \begin{pmatrix} 9 & 3 & -1 & -4 \\ -8 & -2 & 14 & 3 \\ 13 & 2 & -9 & -1 \end{pmatrix} \begin{pmatrix} -1 \\ -4 \\ -30 \end{pmatrix}$$

$(W^{(1)})^T \quad \mathbf{b}^{(1)} \quad (W^{(2)})^T \quad \mathbf{b}^{(2)} \quad (W^{(3)})^T \quad \mathbf{b}^{(3)} \quad U^T \quad \mathbf{c}$

FEEDFORWARD NEURAL NETWORKS: EXAMPLE

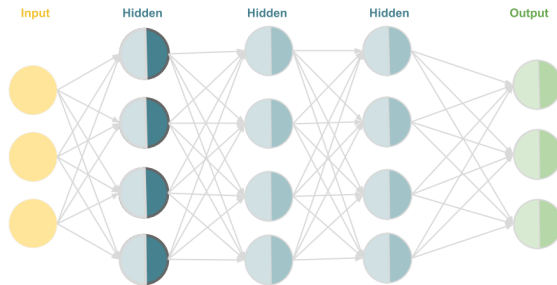


FEEDFORWARD NEURAL NETWORKS: EXAMPLE



$$\begin{bmatrix} 7 \\ 1 \\ -4 \end{bmatrix} \begin{pmatrix} 7*13 + 1*(-9) + (-4)*2 + 5 \\ 7*(-8) + 1*0 + (-4)*3 + (-2) \\ 7*4 + 1*(-1) + (-4)*5 + 2 \\ 7*(-3) + 1*12 + (-4)*7 + 11 \end{pmatrix}$$
$$\mathbf{x} \quad \mathbf{z}_{in}^{(1)} = W^{(1)T} \mathbf{x} + \mathbf{b}^{(1)}$$

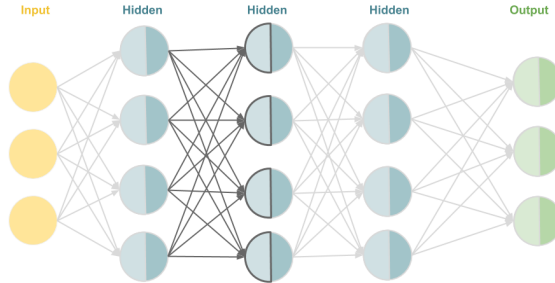
FEEDFORWARD NEURAL NETWORKS: EXAMPLE



7 1 -4	79 -70 9 -26	$\begin{pmatrix} \max(0, 79) \\ \max(0, -70) \\ \max(0, 9) \\ \max(0, -26) \end{pmatrix}$
--	---	---

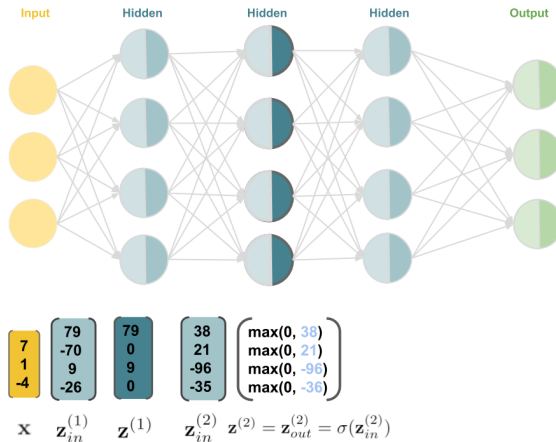
$$\mathbf{x} \quad \mathbf{z}_{in}^{(1)} \quad \mathbf{z}^{(1)} = \mathbf{z}_{out}^{(1)} = \sigma(\mathbf{z}_{in}^{(1)})$$

FEEDFORWARD NEURAL NETWORKS: EXAMPLE

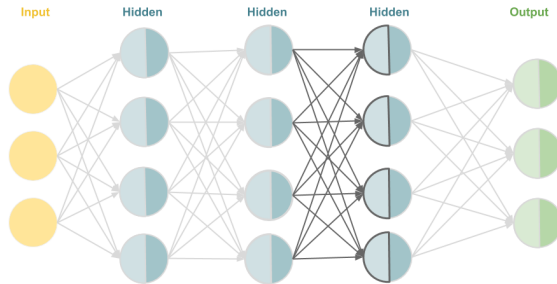


$$\begin{array}{c}
 \begin{bmatrix} 7 \\ 1 \\ -4 \end{bmatrix} \\
 \mathbf{x}
 \end{array}
 \begin{array}{c}
 \begin{bmatrix} 79 \\ -70 \\ 9 \\ -26 \end{bmatrix} \\
 \mathbf{z}_{in}^{(1)}
 \end{array}
 \begin{array}{c}
 \begin{bmatrix} 79 \\ 0 \\ 9 \\ 0 \end{bmatrix} \\
 \mathbf{z}^{(1)}
 \end{array}
 \begin{array}{c}
 \begin{pmatrix} 79*1 + 0*0 + 9*(-4) + 0*1 + (-5) \\
 79*0 + 0*11 + 9*2 + 0*(-14) + 3 \\
 79*(-1) + 0*5 + 9*(-2) + 0*16 + 1 \\
 79*0 + 0*(-9) + 9*(-3) + 0*4 + (-8) \end{pmatrix} \\
 \mathbf{z}_{in}^{(2)} = W^{(2)T} \mathbf{z}^{(1)} + \mathbf{b}^{(2)}
 \end{array}
 \end{array}$$

FEEDFORWARD NEURAL NETWORKS: EXAMPLE

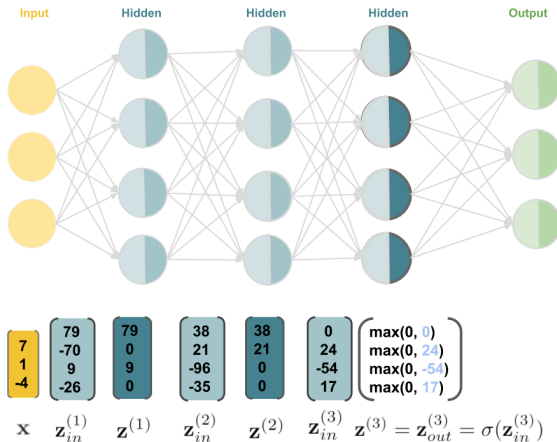


FEEDFORWARD NEURAL NETWORKS: EXAMPLE

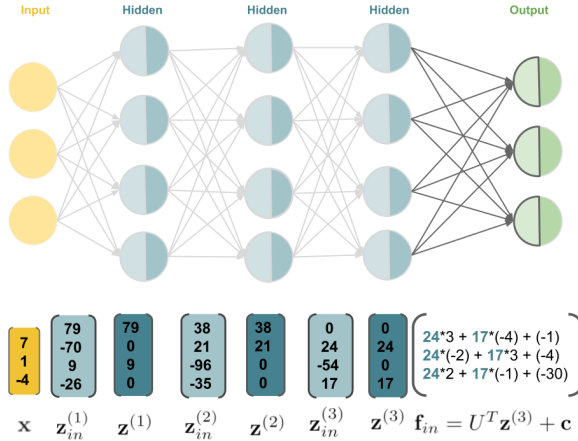


$$\begin{array}{c}
 \begin{bmatrix} 7 \\ 1 \\ -4 \end{bmatrix} \quad \begin{bmatrix} 79 \\ -70 \\ 9 \\ -26 \end{bmatrix} \quad \begin{bmatrix} 79 \\ 0 \\ 9 \\ 0 \end{bmatrix} \quad \begin{bmatrix} 38 \\ 21 \\ -96 \\ -35 \end{bmatrix} \quad \begin{bmatrix} 38 \\ 21 \\ 0 \\ 0 \end{bmatrix} \quad \left(\begin{array}{l} 38*1 + 21*(-2) + 0*(-18) + 0*(-7) + 4 \\ 38*3 + 21*(-4) + 0*8 + 0*0 + (-6) \\ 38*(-2) + 21*1 + 0*21 + 0*5 + 1 \\ 38*2 + 21*(-2) + 0*11 + 0*(-13) + (-17) \end{array} \right) \\
 \mathbf{x} \quad \mathbf{z}_{in}^{(1)} \quad \mathbf{z}^{(1)} \quad \mathbf{z}_{in}^{(2)} \quad \mathbf{z}^{(2)} \quad \mathbf{z}_{in}^{(3)} = \mathbf{W}^{(3)T} \mathbf{z}^{(2)} + \mathbf{b}^{(3)}
 \end{array}$$

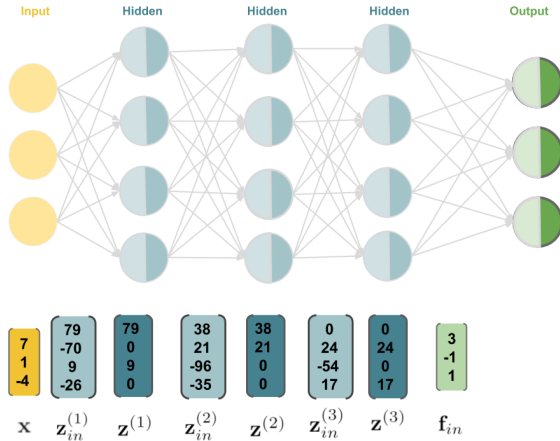
FEEDFORWARD NEURAL NETWORKS: EXAMPLE



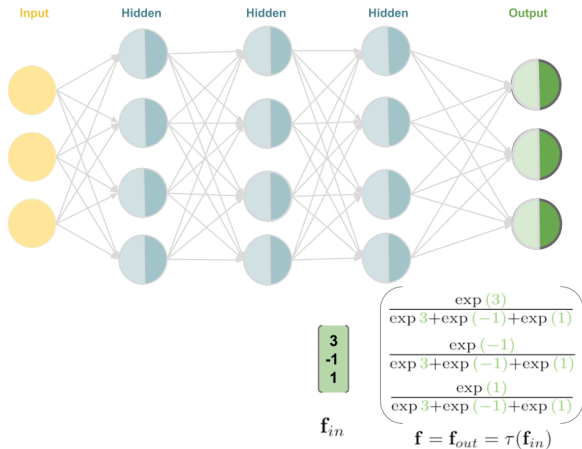
FEEDFORWARD NEURAL NETWORKS: EXAMPLE



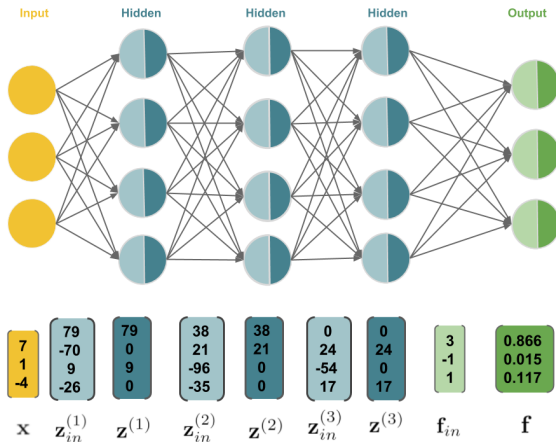
FEEDFORWARD NEURAL NETWORKS: EXAMPLE



FEEDFORWARD NEURAL NETWORKS: EXAMPLE



FEEDFORWARD NEURAL NETWORKS: EXAMPLE



WHY ADD MORE LAYERS?

- Multiple layers allow for the extraction of more and more abstract representations.
- Each layer in a feed-forward neural network adds its own degree of non-linearity to the model.

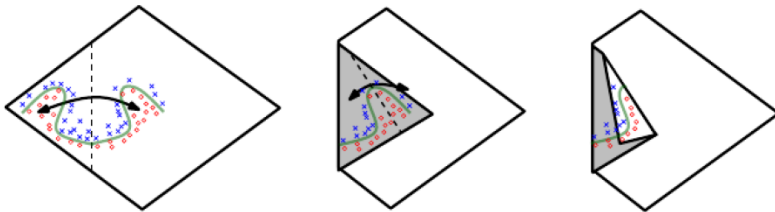


Figure: An intuitive, geometric explanation of the exponential advantage of deeper networks formally (Montúfar et al., 2014).

DEEP NEURAL NETWORKS

Neural networks today can have hundreds of hidden layers. The greater the number of layers, the "deeper" the network. Historically DNNs were very challenging to train and not popular until the late '00s for several reasons:

- The use of sigmoid activations (e.g., logistic sigmoid and tanh) significantly slowed down training due to a phenomenon known as “vanishing gradients”. The introduction of the ReLU activation largely solved this problem.
- Training DNNs on CPUs was too slow to be practical. Switching over to GPUs cut down training time by more than an order of magnitude.
- When dataset sizes are small, other models (such as SVMs) and techniques (such as feature engineering) often outperform them.

DEEP NEURAL NETWORKS / 2

- The availability of large datasets and novel architectures that are capable of handling even complex tensor-shaped data (e.g. CNNs for image data), faster hardware, and better optimization and regularization methods made it feasible to successfully implement deep neural networks.
- An increase in depth often translates to an increase in performance on a given task. State-of-the-art neural networks, however, are much more sophisticated than the simple architectures we have encountered so far.

The term "**deep learning**" encompasses all of these developments and refers to the field as a whole.

REFERENCES



Montúfarr, G., Pascanu, R., Cho, K., & Bengio, Y. (2014). *On the Number of Linear Regions of Deep Neural Networks*.

Universal Approximation

UNIVERSAL APPROXIMATION: WHAT IT SAYS

- **Theorem (informal).** A neural network with a single hidden layer and enough hidden neurons can approximate any continuous function on a bounded input domain to arbitrary precision.
- This holds for (almost) any non-linear activation function σ , and extends trivially to multiple outputs.
- Two important caveats:
 - The theorem only guarantees that suitable weights *exist* — not that training will find them.

UNIVERSAL APPROXIMATION: WHAT IT SAYS / 2

- “Enough hidden neurons” can mean an impractically large number: a network of fixed size can only represent a subspace of all continuous functions.

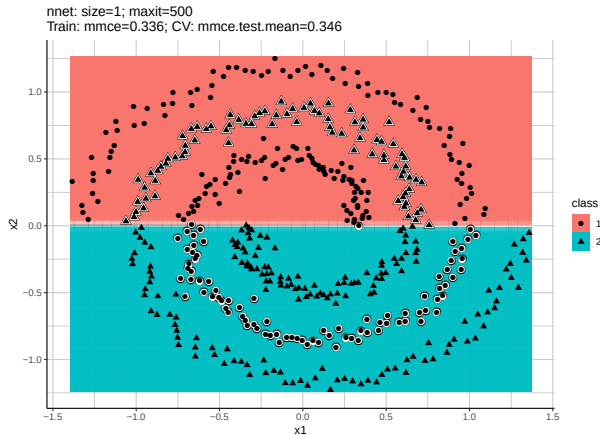
UNIVERSAL APPROXIMATION: THE CATCH

- Universal approximation means approximation error can be driven to (near) zero — so why not always use huge networks?
- Because a highly flexible model class is also prone to **overfitting**: fitting the training data closely does not guarantee good generalization.
- In practice, models of **intermediate flexibility** tend to generalize best; for neural networks, that means a reasonably (not maximally) sized hidden layer.

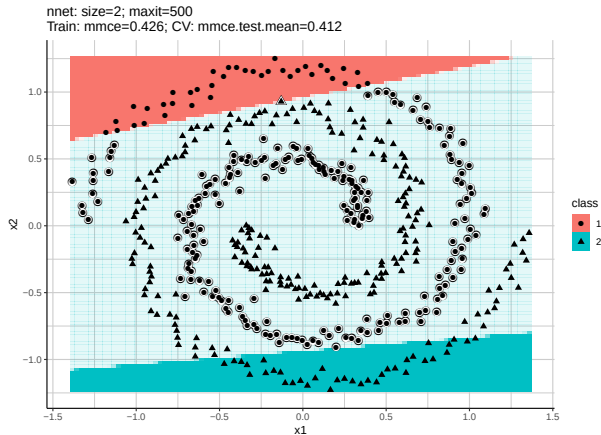
UNIVERSAL APPROXIMATION: THE CATCH / 2

- Let's see this in practice: decision boundaries learned by single-hidden-layer networks of different sizes, at different points during training.

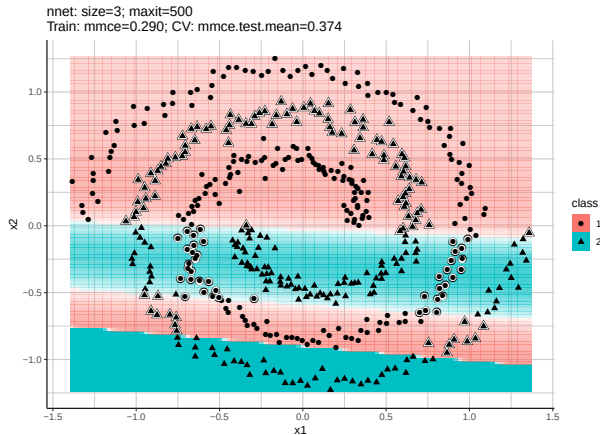
CLASSIFICATION: 500 TRAINING ITERATIONS



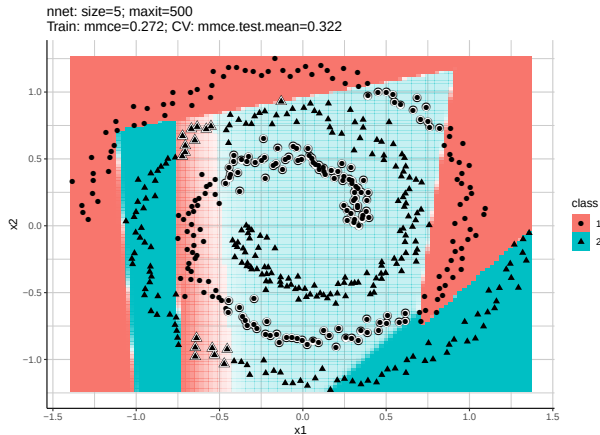
CLASSIFICATION: 500 TRAINING ITERATIONS



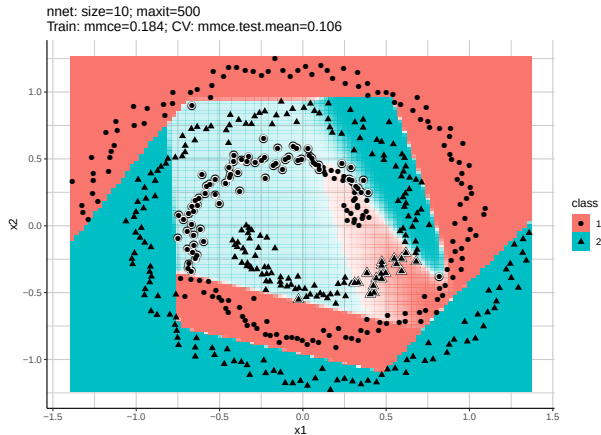
CLASSIFICATION: 500 TRAINING ITERATIONS



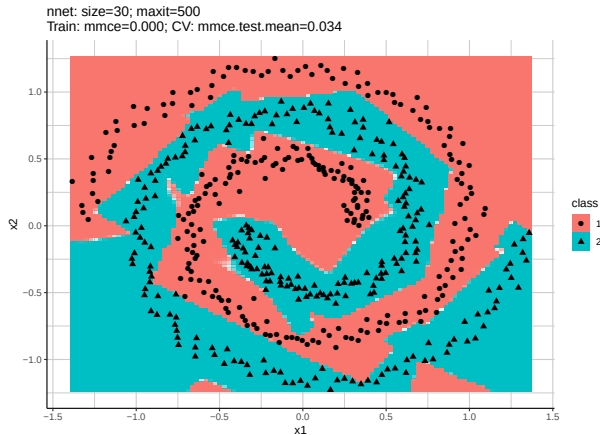
CLASSIFICATION: 500 TRAINING ITERATIONS



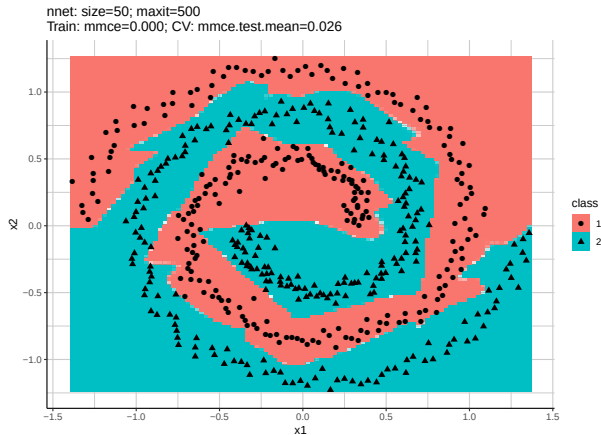
CLASSIFICATION: 500 TRAINING ITERATIONS



CLASSIFICATION: 500 TRAINING ITERATIONS



CLASSIFICATION: 500 TRAINING ITERATIONS



LICENSE & ATTRIBUTION

- This deck is adapted from **Introduction to Deep Learning (I2DL)**, Prof. Dr. Bernd Bischl, Statistical Learning and Data Science (SLDS), LMU Munich.
https://github.com/slds-lmu/lecture_i2dl
- Licensed **CC BY 4.0**:
<https://creativecommons.org/licenses/by/4.0/>
- Content has been merged, reordered, and consolidated from the original per-topic decks into this single session deck; not used verbatim.